

“ChatGPT Suggested This Fix and It Seems to Work”: What Code Comments Reveal about Human-AI Collaboration

Anonymous Author(s)

Abstract

Developers are increasingly incorporating large language models (LLMs) such as ChatGPT, Copilot, Cursor, and Claude into everyday workflows. While most studies evaluate LLM outputs in isolation, how developers refine and adapt these suggestions for real project integration remains underexplored. In this paper, we conduct an empirical analysis of 12,043 GitHub code comments that explicitly reference the use of LLMs, providing direct evidence of their involvement in real-world software development. We examine how developers integrate LLM suggestions, the types of contributions these models make, and how LLM-generated code evolves through subsequent developer intervention.

Through open-coding of 500 unique comments and subsequent large-scale annotation of 12,043 comments using two LLM classifiers aggregated through the Dawid-Skene Expectation-Maximization (DS-EM) framework (RQ1), we find that developers primarily use LLMs for *code implementation*, followed by *code enhancement*, *debugging*, *documentation*, and *testing*. Developers use LLMs as collaborative tools to generate new ideas, explore alternatives, and refine solutions. Commit-level analysis (RQ2) show that subsequent revisions often involve *refactoring*, *feature extension*, and *bug fixing*, indicating sustained human oversight in improving LLM-generated code. A longitudinal analysis of Github commits from Dec 2022-Jul 2025 (RQ3) further reveals a shift from direct code generation toward greater emphasis on enhancement, documentation, and conceptual support tasks, reflecting the evolving integration of LLMs into everyday software development.

ACM Reference Format:

Anonymous Author(s). 2025. “ChatGPT Suggested This Fix and It Seems to Work”: What Code Comments Reveal about Human-AI Collaboration. In . ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Large language models (LLMs) such as GitHub Copilot, ChatGPT, Cursor, and Claude have become integral to modern software development. Developers now frequently rely on these models to generate code, explain [3], refactor existing modules [1], and produce documentation [10]. This widespread adoption is reshaping the nature of programming, from an exclusively human activity to a collaborative process involving both human and AI contributions [37]. As these tools become embedded within daily workflows,

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference’17, Washington, DC, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

```
windows.forEach((win) => {  
  // For some reason just using the win object had cases  
  // where the window size wouldn't change for  
  // the non-active window. ChatGPT suggested this fix  
  // and it seems to work, although I have no idea  
  // why it would be any different than just using the win  
  // object.  
  const targetWindow =  
    BrowserWindow.fromId(win.id);  
  
  [...]  
});
```

ChatGPT suggests a fix.

Developer applies the AI fix but ponders why it works when the previous solution did not.

Figure 1: Example of a Real-World Code Comment Reflecting Human-AI Collaboration.

understanding how developers actually use LLMs in practice has become an essential question for software engineering research [35].

Most prior studies on LLM-assisted programming rely on controlled experiments or benchmark evaluations to measure model performance in code generation and related tasks [3, 35, 39]. Benchmark-based and user studies have analyzed tools such as GitHub Copilot, Claude, and ChatGPT, as well as models like Llama, Qwen, and Gemini, primarily focusing on developer productivity, usability, code quality, error correction, and problem-solving processes [9, 27]. While, these studies offer useful technical insights, they largely reflect artificial or short-term evaluation settings. How developers actually use, adapt, and integrate LLM outputs in real projects remains largely unexplored, particularly how such code changes and matures once integrated into software repositories.

To address this gap, we collect and examine artifacts from real GitHub projects, specifically those that reveal how large language models are used within development workflows. By focusing on these in-situ artifacts, we can observe how developers employ, interpret, and modify LLM-generated outputs in different programming or task contexts. Although prior work has examined commits, code reviews, pull requests, and issue discussions [6, 11, 17, 18], these efforts mainly capture higher-level collaboration and coordination activities. The finer-grained interactions, i.e., how developers engage with and adapt model-generated code within source files, remain comparatively underexplored.

Among these finer-grained artifacts, code comments provide a particularly valuable lens for analysis [41]. Comments capture developers’ reasoning in context: they describe intent, limitations, and uncertainty, and often include self-admitted acknowledgements of model involvement (e.g., “generated by ChatGPT,” “suggested by Copilot,” “AI-generated, please review”). Unlike commits or documentation, comments are co-located with code, offering a fine-grained evidence of how developers interpret and modify

LLM-generated content. Figure 1 shows an example of such comments observed in GitHub repositories, illustrating how developers describe, critique, or justify AI-assisted code within their projects.

This study investigates how developers use, reflect on, and modify LLM-generated code as noted in code comments in open-source GitHub projects. We focus on developer-authored, self-admitted code comments that explicitly reference LLM use. These comments, along with associated commits, serve as in-situ evidence of how developers collaborate with LLMs during real-time development. By analyzing these artifacts, we aim to uncover how LLMs are used in everyday programming and how developers adapt or refine their generated code over time. To guide this investigation, we pose the following research questions:

RQ1: How do developers integrate LLMs into their real-world software development workflows, and what forms of contribution do these models make?

To address this question, we manually annotated 500 unique code comments that explicitly referenced LLM usage and extended the analysis to 12,043 comments using LLM-based annotation consolidated through the Dawid-Skene (DS) Expectation-Maximization (EM) framework [8]. We categorized each instance along two dimensions: **Task Type**, describing the software engineering activity supported by the LLM (e.g., code implementation, enhancement, testing, documentation, or debugging); and **LLM Contribution Type**, representing how the model assisted developers (e.g., implementation, knowledge and concept support, or artifact generation). Using both quantitative and qualitative analyses, we find that LLMs are most often used for code implementation and conceptual guidance, positioning them as active collaborators within real-world software development workflows.

RQ2: How do developers' commit-level activities reflect the software engineering actions involved in integrating and modifying LLM-assisted code?

Each annotated comment was linked to its corresponding *introductory commit*. We also linked the *first change commit*, which refers to the earliest subsequent commit that modifies, extends, or deletes the LLM-assisted code introduced in the introductory commit. Using semantic clustering of commit messages with HDBSCAN [32], we identified 11 integration and 9 modification activity types. We found that developers most often use LLMs for feature development, project initialization and setup, and code refactoring. During modification, the dominant activities were code refactoring and feature development, followed by bug fixing and repository management.

RQ3: How has developers' LLM usage behavior evolved over time?

We performed a longitudinal analysis of GitHub commits from December 2022 to July 2025 to analyze the evolution of LLM-assisted activities at two levels of granularity from RQ1: a) **Task Types** and b) **LLM Contribution Types**. Over time, *code implementation* remained the dominant activity but gradually declined in relative frequency, while *enhancement* and *documentation* gained prominence. At the contribution level, *implementation* continued to lead, while *knowledge-seeking* behavior increased most rapidly, suggesting that developers are progressively engaging LLMs for conceptual reasoning, design exploration, and informed decision-making rather than solely for code generation.

Contributions. This paper makes the following contributions:

- **Dataset.** A curated dataset of 12,043 GitHub code comments explicitly referencing LLMs (e.g., ChatGPT, Copilot, Claude, Cursor), spanning December 2022–July 2025 and 5,721 repositories, each linked to corresponding commits and repository context.
- **Annotation.** A two-stage pipeline combining 500 manually annotated comments and large-scale LLM-assisted labeling, consolidated through probabilistic aggregation to ensure label consistency.
- **Commit-level Analysis.** A large-scale HDBSCAN-based semantic clustering of *introductory* and *first change* commits linked to LLM-referenced code, identifying 11 integration and 9 modification activity categories.

DATA AVAILABILITY. We provide the annotation guidelines, dataset, and relevant codes to enable replication of our study at [2].

2 Methodology

We adopt a multi-stage, mixed-method research design that combines large-scale data mining, human qualitative coding, probabilistic multi-model annotation, and semantic clustering. Figure 2 provides an overview of the research methodology. We begin by collecting code comments that explicitly reference LLM usage, covering the period from December 2022 (when ChatGPT was first released) to July 2025. The annotation process consists of two phases: (i) manual human annotation and (ii) LLM-based annotation with probabilistic aggregation. RQ1 is examined using this curated and consolidated dataset. In the subsequent stage, we extract two categories of commits: (i) those introducing the identified LLM-referenced comments and (ii) those reflecting later modifications to the same code files. In RQ2, we apply HDBSCAN clustering to these commits to uncover recurrent patterns and underlying motivations driving developers' modifications and refinements of LLM-assisted code over time. In RQ3, we apply time-series analysis on our derived categories in RQ1 to understand the evolving nature of trends.

2.1 Data Collection

We focus on Python and JavaScript, the two most widely used languages in today's open-source ecosystem [46]. Python emerged as the most used language on GitHub in 2024 amid the generative-AI surge [16], while JavaScript has remained a consistent leader in developer activity [15].

2.1.1 Extraction of LLM-Referenced Comments. To identify explicit instances where developers referenced the use of LLMs and AI coding assistants as comments in source code, we queried GitHub using the Code Search API with a systematically constructed set of search query templates, as described next. Each template was generated by combining three sets of phrases:

- 16 large language models (e.g., *ChatGPT*, *GPT*, *Copilot*, *Claude*, *Gemini*, *Llama*, etc).
- 6 action verbs implying generative behavior (e.g., *generated*, *suggested*, *created*, *written*, *authored*, and *assisted*); and
- 4 connector terms indicating attribution (e.g., *by*, *from*, *with*, and *using*).

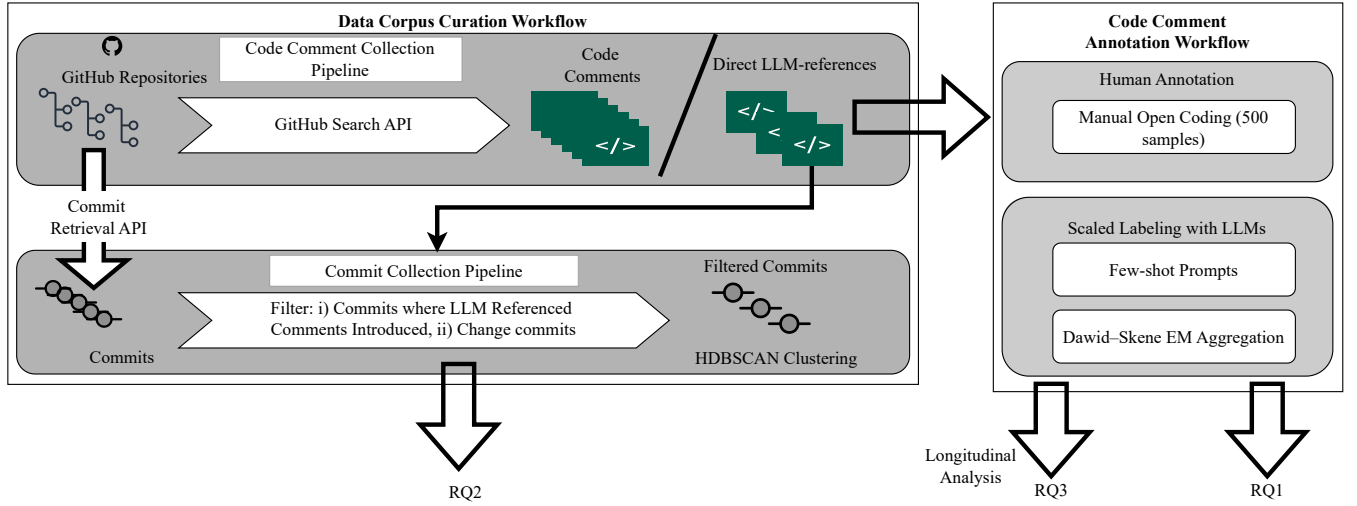


Figure 2: Overview of methodology.

We applied two complementary query patterns to capture both forward and reversed attributions. The first pattern, ($A \times B$), matched direct expressions such as “ChatGPT generated” or “Claude suggested,” while the second pattern, ($C \times B \times A$), captured reversed constructions such as “generated by ChatGPT” or “written with Copilot”. The first pattern consists of 96 search queries (16×6) and the second pattern consists of 384 queries ($16 \times 6 \times 4$) resulting to a total of 480 search queries. Executing these queries yielded 37,234 source files (24,346 Python and 12,888 JavaScript). To ensure that matches corresponded to explicit developer attributions, we applied Abstract Syntax Tree (AST) analysis using *Tree-sitter* on the retrieved source files [44]. This step verified that each identified phrase occurred within code comment regions, filtering out incidental string occurrences within executable code or documentation.

After filtering, we collected 10,165 source files containing 12,043 comment-level matches, which formed the initial corpus.

2.1.2 Linking Code Comments to Associated Commits. To examine what development activities developers perform on LLM-referenced code both during integration and after integration into software projects, we linked identified comments to two types of commits: (i) the **introductory commit**, marking when the comment first appeared in the repository, and (ii) the **first change commit**, representing the earliest subsequent modification to the same file.

We retrieved commit metadata for 10,165 source files using the GitHub REST API, collecting commit SHA, author, date, message, and corresponding patch. Commits were successfully obtained for 8,052 files, while others were unavailable due to deletion or repository relocation. Each comment was then matched to the relevant commit patches to determine its introductory and first change commits. In total, we identified **introductory commits** for 8,177 comments and **first change commits** for 3,039 comments. The number of comments is higher than the number of files since some files contain multiple LLM-referenced comments, each was linked to its own introductory commit.

2.1.3 Summary of Data Collection. We collected 10,165 source files containing 12,043 LLM-referenced code comments. From these, we identified 8,177 introductory and 3,039 first change commits. A portion of the files could not be retrieved, likely due to deletion or repository relocation. Python accounted for 62.95% and JavaScript for 37.05% of the corpus.

2.2 Data Annotation and Validation

2.2.1 Selection. A stratified random sample of 500 unique comments (balanced by language) was selected for manual coding. Two annotators (with 6+ years of programming experience) independently examined each comment to answer: “What developer task did the LLM assist with, and how?”. Each comment was annotated along two dimensions:

- (1) **Task Type**, describing the developer activity (e.g., implementation, debugging, documentation, testing, or refactoring); and
- (2) **LLM Contribution**, indicating how the model supported the task (e.g., code generation, suggestion, or auxiliary artifact creation).

They were given the following instructions:

Please review the annotation instructions carefully. Then, annotate each comment along two dimensions: **Task Type** and **LLM Contribution Type**. Assign multiple codes if needed. If an LLM is mentioned but not actually used, mark it as *False Positive* in both dimensions. Use open coding to identify initial patterns, and during axial coding, **select the single most applicable category** that best represents each comment to merge related patterns into broader themes, forming the final taxonomies.

An iterative grounded process followed the annotation. The annotators first performed open coding to identify recurring themes, then refined them into a taxonomy of task and contribution categories through axial coding. Disagreements were resolved through several rounds of discussion, resulting in a consistent coding scheme.

During annotation, a substantial number of false positives were detected. We observe instances where LLM-related terms appeared incidentally in comments (e.g., someone named Claude commented: “written by Claude Pageauoment”). To maintain precision, these cases were excluded, and additional samples were drawn and annotated until a sufficient volume of valid, LLM-referenced comments was achieved. A total of 173 false positives were removed over 3 rounds of annotation. This process expanded the manually verified set beyond the initial 500 instances.

After axial coding, we identified six **Task Types**: *Code Implementation* (362/500), *Code Enhancement* (20/500), *Bug Identification & Fixing* (26/500), *Testing* (23/500), *Documentation* (16/500), and *Generic Mention and Indeterminate Actions* (97/500); and four **LLM Contribution Types**: *Implementation* (387/500), *Knowledge and Concept Support* (50/500), *Artifact Generation* (15/500), and *Generic Mention and Indeterminate Actions* (48/500).

While the label *Generic Mention and Indeterminate Actions* appears in both **Task Type** and **LLM Contribution Type** categories, it serves different analytical purposes in each dimension. In **Task Type**, it denotes instances where the developer’s activity could not be identified, whereas in **LLM Contribution Type**, it indicates that the model’s role was unclear or unspecified. This distinction is further supported by their differing frequencies in the annotated data. For example, a comment such as “Suggested by CoPilot”, indicates that the a suggestion was taken from LLM, but it does not reveal any specific task category.

Inter-annotator agreement was assessed separately for both dimensions using Cohen’s kappa for multiclass classification. The results yielded $\kappa = 0.779$ for *Task type* and $\kappa = 0.832$ for *LLM Contribution type*, indicating substantial consistency (≥ 0.6). The validated corpus provided a reliable empirical basis for the taxonomy of developer-LLM interactions and served as calibration data for subsequent large-scale annotation.

2.3 Scaling Annotation with LLMs

Since manually annotating rest of the dataset of 11370 code comments is not feasible, we used **few-shot prompting strategy** integrated with **Dawid-Skene (DS) Expectation-Maximization (EM)**-based label aggregation framework [8, 50], to extend annotation to the full corpus. Given annotations on the same data by multiple LLMs, the DS-EM method infers the most probable true label by modeling annotator reliability and agreement, allowing multiple LLM outputs to be combined into a single consensus label. It has been widely adopted in NLP for combining crowd-sourced or LLM annotations [13, 20, 23, 25, 45, 52].

2.3.1 LLM-Based Labeling. We employed two open-weight models, *gpt-oss:20b* [38] and *mistral-small-3.2* [33], to enhance robustness and mitigate model-specific bias. *gpt-oss:20b* is trained for both general and code-oriented language understanding, while *mistral-small-3.2:24b* is a lightweight model optimized for efficient reasoning across code and natural text. We used ollama platform and set temperature 0 for both models. By combining outputs from these distinct architectures, we obtain probabilistically aggregated labels that better capture the consensus between models and reduce variance arising from model-specific behaviors.

First, since many comments in our dataset were repetitive (e.g., “generated by ChatGPT”), we removed duplicates, which yielded 6,867 unique comments. Then we used two structured prompts to guide the process: (1) **Task Type** prompt, which categorized each comment according to the 6 software development activity identified during axial coding; and (2) **LLM Contribution Type** prompt, which identified how the LLM contributed based on 4 categories derived from the same coding process. If a comment did not fit any category, we instructed the LLMs to label it as *False Positive*. The prompt templates are available in the replication package [2].

Each comment was independently annotated by both LLMs, producing parallel label sets. In 10 cases, the LLMs (one or both) did not generate output in the correct format. We excluded these instances from our dataset. Of the remaining, 6,857 cases, for **Task Type** annotation, the annotated labels by *gpt-oss:20b* and *mistral-small-3.2:24b* were same for 4,399 instances. For 2,458 comments, the LLMs predicted different labels. For **LLM Contribution Type** annotations, both LLMs predicted same labels for 3,848 instances and in 3,009 cases they predicted different labels.

These labels were subsequently consolidated using the Dawid-Skene Expectation-Maximization (DS-EM) aggregation procedure [8], which we describe next.

2.3.2 Dawid-Skene Expectation-Maximization (DS-EM) Aggregation. The Dawid-Skene EM algorithm jointly estimates (1) the latent true label for each instance and (2) the reliability of each annotator—in this case, the two LLM classifiers [8, 50]. Each item i has a latent class $Y_i \in \{1, \dots, K\}$ with prior $\pi_k = P(Y = k)$. Observed labels $\ell_i^{(A)}$ and $\ell_i^{(B)}$ follow confusion matrices $C^{(A)}$ and $C^{(B)}$, assuming independence: $P(\ell_i^{(A)}, \ell_i^{(B)} | Y_i = k) = C_{k, \ell_i^{(A)}}^{(A)} C_{k, \ell_i^{(B)}}^{(B)}$. The EM procedure alternates between estimating posteriors $p_i(k) \propto \pi_k C_{k, \ell_i^{(A)}}^{(A)} C_{k, \ell_i^{(B)}}^{(B)}$ and updating priors and confusion matrices with Dirichlet smoothing [14]. Diagonal priors are biased ($\alpha_{kk} > \alpha_{k\ell}$) to encode higher annotator accuracy.

Gold-standard labels initialize π and C , anchoring the EM process. Training continues until convergence ($\Delta\mathcal{L} < 10^{-6}$) or 100 iterations. Hyperparameters: $\alpha_{\text{diag}} = 2.0$, $\alpha_{\text{off}} = 0.5$, $\gamma_\pi = 10^{-3}$. Each instance yields a posterior vector $p_i(\cdot)$, hard label $\hat{y}_i = \arg \max_k p_i(k)$, and confidence margin.

This setup was applied to both **Task Type** and **LLM Contribution Type** annotations.

DS-EM identified a total of 802 comments as *False Positives*. We excluded these instances from our dataset. To get the analysis over the full dataset containing 12,043 comments, we reverse mapped rest of the comments. We found 1,516 false positives in the full dataset. Furthermore, 10 comments previously removed due to LLM output errors were mapped to 99 instances. After excluding those cases, the final aggregated corpus comprised 10,428 comments, categorized as follows: **Task types**: *Code Implementation* (3,756), *Code Enhancement* (367), *Bug Identification & Fixing* (188), *Testing* (196), *Documentation* (390), and *Generic Mention and Indeterminate Actions* (5,531); and **LLM Contribution types**: *Implementation* (2,104), *Knowledge and Concept Support* (573), *Artifact Generation* (377), and *Generic Mention and Indeterminate Actions* (7,374).

2.4 Semantic Clustering of Commit Messages

To identify developers’ intentions or reasons for using LLMs in their workflow and later changing them, we analyzed the semantics of their corresponding commit messages. Using the sets of **8,177 introductory** and **3,039 first change commits**, we examined how developers describe the introduction and subsequent modification of LLM-assisted code.

To ensure the consistency of commit data, we removed the commits associated with *False Positive* comments (discussed in 2.3). We also observed that some commits had unusual commit date (i.e., “2027-01-15T23:46:50Z”) despite clear mention of LLM usage in the code comments. It is likely because commit dates can be set manually. We excluded them for this analysis and obtained 6,912 **introductory commits** and 2,948 **first change commits**, each associated with a categorized comment from our comment dataset.

2.4.1 Identifying development activity from Introductory Commits. We used Hierarchical Density-Based Spatial Clustering of Applications with Noise (HDBSCAN) [32] to group semantically similar commit messages. HDBSCAN can identify clusters of different densities and label uncertain points as noise, without requiring a predefined number of clusters. This method has been adopted in software engineering research, including for analyzing developer comments [26].

Each commit message underwent text normalization, lemmatization, and vector embedding using the *all-MiniLM-L6-v2* model [42]. After manual parameter sweeping, we selected cosine distance ($\epsilon = 0.20$), minimum cluster size ($minPts = 10$), and the *leaf cluster selection*. This procedure produced 58 distinct clusters.

We interpreted and labeled each cluster through a structured process that combined keyword heuristics with human review. We first built a keyword dictionary based on existing commit-message taxonomies [22], including categories such as *feature*, *fix*, *refactor*, *documentation*, and *build*. We then expanded the dictionary by extracting additional keywords from our dataset using TF-IDF weighting and part-of-speech filtering to capture frequent action-oriented terms such as *security*, *revert*, *dependency*, *style*, and *optimization*.

For each cluster, we matched the most significant tokens against the keyword dictionary using a weighted scoring function that emphasized key verbs and nouns such as *add*, *fix*, *refactor*, and *upgrade*. We then assigned preliminary labels and refined them through manual review. One author examined representative messages per cluster to ensure conceptual accuracy and adjust the labels as needed. Finally, we merged related clusters into broader categories [12]. For example, *Refactoring*, *Code Cleanup*, and *Refactor After Testing* were combined into a single cluster labeled *Code Refactoring*.

After completing the initial labeling and merging phase, a second annotator independently reviewed the consolidated clusters to verify label consistency and address remaining ambiguities. Our clustering identified 11 categories of developer intent involving LLM assistance. These categories are: *Feature Development*, *Project Initialization and Setup*, *Code Refactoring*, *Documentation*, *Repository Management*, *Security and Compliance*, *Release and Deployment*, *Tooling*, *Bug Fixing*, *LLM/AI Work*, and *Testing and Verification*.

2.4.2 Identifying Modification Activities on LLM Assisted Code. To get insight of why developers are modifying LLM aided files after integrating them into the projects, we did a similar HDBSCAN clustering described in 2.4.1 on the commit messages from 2,948 **first change commits**. After a manual parameter sweep, we used cosine distance ($\epsilon = 0.25$), minimum cluster size ($minPts = 6$) along with *leaf cluster selection*. This process resulted in 70 different clusters.

We followed the same multi-stage labeling and validation process described in Section 2.4.1, combining automated keyword extraction with targeted human review. After merging similar clusters into broader, interpretable groups [12], a second annotator verified label consistency and resolved remaining ambiguities. This process resulted in nine clusters representing the main reasons developers modify LLM-aided code. These clusters include: *Code Refactoring*, *Feature Development*, *Bug Fixing*, *Repository Management*, *LLM/AI Work*, *Testing and Verification*, *Documentation*, *Security and Compliance*, and *Logging*.

2.5 Longitudinal Analysis for Task Type and LLM Contribution Type

We aggregated monthly annotation counts for both **Task Type** and **LLM Contribution Type** based on each comment’s **introductory commit** timestamp between December 2022 and June 2025, forming a continuous longitudinal series. Categories labeled as *Generic Mention and Indeterminate Actions* were excluded to reduce semantic noise and isolate interpretable actions. Each series was then normalized by its monthly total, yielding proportional rather than absolute frequencies to enable cross-category comparison [40].

To attenuate short-term fluctuations, we applied a six-month rolling mean smoother, providing temporal continuity while preserving structural variation [4]. Sharp spikes and irregularities were further corrected using an Interquartile Range (IQR)-based anomaly cap, clipping values outside the $[Q_1 - 1.5 \times IQR, Q_3 + 1.5 \times IQR]$ range [5, 49]. This adjustment minimized distortion from one-off surges in specific categories.

From the smoothed and corrected series, we computed descriptive statistics for each label: *mean*, *standard deviation*, *lag-1 autocorrelation* (ρ_1), *linear trend slope per month* (β), and the corresponding *p-value* assessing the statistical significance of β . These metrics capture the stability, variability, and directionality of LLM activity over time [4]. All computations were performed on the corrected normalized data.

3 Results and Discussion

In this section, we discuss the findings of each research question.

3.1 RQ1: How do developers integrate LLMs into their real-world software development workflows, and what forms of contribution do these models make?

To study how developers use LLMs, we look at two aspects: what kinds of tasks LLMs are used for and how they help developers during those tasks. Thus, **RQ1.A** identifies the types of development

Table 1: Developer Level Task Distribution of LLM Usage in GitHub Code Comments

Category	Description	Example	Annotated	Full Dataset
Code Implementation	LLM is helping the developer in implementation including programming tasks, database query tasks and more.	"This function was created by Claude AI."	362 (89.82%)	3756 (76.7%)
Documentation	Comment, Docstring or Documentation is supported by LLMs	"Description: this file contains test cases generated by copilot."	16 (3.97%)	390 (7.96%)
Code Enhancement	LLM is helping to improve performance or readability of existing code	"This function was generated using ChatGPT with the prompt: Improve the delete_task function [...]"	20 (4.96%)	367 (7.49%)
Testing	LLM is helping with testing by test case generation/suggesting test techniques	"Description: this file contains test cases generated by copilot."	23 (5.71%)	196 (4%)
Bug Identification & Fixing	LLM is helping to find/fix bugs	"Suggested by ChatGPT - Fixes PyCharm backend crash"	26 (6.45%)	188 (3.84%)
		Count	403	4897

‡ -Generic Mention and Indeterminate Actions excluded.

tasks involving LLMs, and **RQ1.B** examines how these models contribute to developers' workflows.

RQ1.A: (In-situ Tasks). What types of development tasks do developers use LLMs for?

Our analysis of 500 manually annotated code comments identifies six categories of LLM-assisted development activities. These include a *Generic / Indeterminate* category (97/500, 19.40%) representing cases where LLMs were mentioned without a clear task context. This category was excluded from subsequent quantitative analysis. The remaining five task-specific categories are: *Code Implementation* (362/403, 89.82%), *Code Enhancement* (20/403, 4.96%), *Bug Identification & Fixing* (26/403, 6.45%), *Testing* (23/403, 5.71%), and *Documentation* (16/403, 3.97%). Table 1 summarizes the frequency and examples in each category.

Code Implementation involves developers using LLMs to generate functional code that becomes part of production repositories. For example, comments such as "This function was created by Claude AI." illustrate model-driven code generation embedded within active projects. *Code Enhancement* represents developers using LLMs to refactor or optimize existing code, for instance, "This function was generated using ChatGPT with the prompt: 'Improve the delete_task function with better error handling and improved readability.'", highlighting how models assist in iterative improvement during development.

Developers also used Generative AI for bug fixing, code quality assurance, and documentation tasks. *Bug Identification & Fixing* includes AI-suggested fixes and patches such as "Suggested by ChatGPT - Fixes PyCharm backend crash.," demonstrating their role in detecting and resolving issues within active codebases. *Testing* involves model-generated or suggested test cases integrated directly into testing pipelines. *Documentation* encompasses LLM-generated docstrings, inline comments, and file-level descriptions, e.g., "Description: this file contains test cases generated by Copilot." Finally, *Generic Mentions & Indeterminate Actions* capture cases where LLM usage is acknowledged without additional context or specification.

Extending this taxonomy to the full dataset of 10,428 instances using DS-EM framework, we observed that a substantial portion of the comments were categorized as *Generic Mentions and Indeterminate Actions* (53.04%). After excluding these instances, a clearer distribution of activity types emerged. Among the remaining 4,897 task-specific cases, *Code Implementation* (3756/4897, 76.7%) remains the most dominant activity. However, the relative presence of *Code*

Enhancement (20/403, 4.96% to 367/4897, 7.49%) and *Documentation* (16/403, 3.97% to 390/4897, 7.96%) increases, indicating that developers are not only using LLMs for code generation but also for improving performance, and generating explanations.

RQ1.B: (Forms of Assistance). How do LLMs assist developers in performing these tasks in practice?

Building on **RQ1.A**, this question investigates how developers make use of LLMs during actual programming activities. Specifically, it explores the *form of assistance* these models provide within software development workflows.

The manual coding of 500 code comments revealed three primary forms of assistance: *Implementation* (387/452; 85.62%), *Knowledge and Concept Support* (50/452; 11.06%), and *Artifact Generation* (15/452; 3.32%). The rest were *Generic Mention and Indeterminate Actions* (48). Similar to RQ1.A, we extended this taxonomy for the full dataset of 10,428 comments. 7,374 (70.71%) comments were labeled as *Generic Mention and Indeterminate Actions*. Table 2 shows the distribution of LLM contribution types, along with examples, in both the manually annotated and the full dataset of 3,054 comments.

The findings indicate that the vast majority of interactions involve *Implementation*, where developers use LLMs to produce executable code or complete functional logic that becomes part of a repository. In this mode, models act as direct contributors rather than advisory tools. Comments such as "Validation fully written by Copilot based on error state name" or "Function generated by ChatGPT for sorting users by last activity" exemplify this behavior.

A subset of comments captures *Knowledge and Concept Support* interactions. In the full dataset, excluding the *Generic Mention and Indeterminate Actions*, 18.76% comments indicate developers seeking knowledge, idea or suggestions from the LLMs. Here, developers use LLMs as on-demand advisors to acquire insights, clarify design options, or refine implementation strategies. For instance, the comment "Using bcrypt for password hashing (ChatGPT suggested secure hashing practices)" illustrates the role of LLMs as conceptual partners offering reasoning or design advice. We also observe algorithmic knowledge seeking such as - "Class to get models in DML two stages suggested by LLM" in the comments. A closer look on the 50 manually comments shows diverse areas where developers leveraged LLMs to obtain conceptual or technical guidance. Interactions involved algorithm selection and data structure design (12/50), where developers sought help identifying efficient computational approaches or representation strategies.

Table 2: Distribution of LLM Contribution Type in Open Source Software

Category	Description	Example	Annotated	Full Dataset
Implementation	LLMs are providing direct implementation and generating code	"Validation fully written by copilot based on error state names"	387 (85.62%)	2,104 (68.89%)
Knowledge and Concept Support	LLMs are providing suggestionidea or providing knowledge based on developer query	"Using bcrypt for password hashing (ChatGPT suggested secure hashing practices"	50 (11.06%)	573 (18.76%)
Artifact Generation	LLMs are generating artifacts such as images, texts, config files, etc	"This is a ChatGPT generated list of devices"	15 (3.32%)	377 (12.34%)
Total			452	3054

‡ -Generic Mention and Indeterminate Actions excluded.

Others focused on performance optimization and debugging (8/50), using LLMs to reason about runtime behavior or resolve implementation issues. A smaller number of cases involved clarifying programming syntax, framework functions, or API behavior (6/50), showing that developers also use LLMs for environment-specific questions. The remaining comments reflected broader guidance and exploratory reasoning, such as seeking design rationale, confirming best practices, or evaluating alternative architectural choices.

The third most prevalent type of contribution is *Artifact Generation*. In these cases, models generate supplementary artifacts that go beyond code, such as documentation, test descriptions, or textual artifacts supporting the development process. Comments like "This is a ChatGPT-generated list of device" represent this category.

RQ1 Summary and Discussion. Developers use LLMs as active collaborators across multiple stages of software development. Most interactions involve *Code Implementation*, where models generate production-level code, confirming their role as direct contributors. *Code Enhancement* and *Documentation* reflect developers' use of LLMs for refining existing code and producing descriptive artifacts. *Knowledge and Concept Support* interactions show that developers also treat LLMs as advisory systems, consulting them for design decisions, debugging, and best practices. Although *Artifact Generation* occurs less frequently, it highlights LLMs' role in creating supporting materials such as *configs* and test templates.

3.2 RQ2: How do developers' commit-level activities reflect the software engineering actions involved in integrating and modifying LLM-assisted code?

In RQ2, we examine commit-level evidence (i.e., by linking each identified code comments in RQ1 to: i) the *introductory commit* where the comment first appeared, and ii) the first *change commit* reflecting its earliest modification) to understand the software engineering activities developers perform when integrating and later modifying LLM-assisted code within the broader development workflow. We address two complementary aspects: **RQ2.A**, which investigates activities during the introduction of LLM-assisted code, and **RQ2.B**, which explores activities during its first modification after inclusion.

RQ2.A: (Commit-level Tasks.) What type of development activities characterize the introduction of LLM-assisted code into software projects?

Table 3 summarizes the 11 development activities associated with the introduction or integration of LLM-assisted code into software projects, derived from clustering the *introductory commit messages*

Table 3: Development Activities When Integrating LLM-Assisted Code (Based on Introductory Commit Messages)

Development Activity	N	Example Introductory Commit Message
Feature Development	974	"user profile added and user slice created"
Project Initialization and Setup	723	"Initial commit: FileAnalyzer implementation"
Code Refactoring	382	"Cleaned up code"
Documentation	149	"Added comments for human and AI code"
Repository Management	107	"transformed to new git online repo"
Security and Compliance	100	"feat: add new rule 'no-hardcoded-secrets' "
Release and Deployment	98	"stable release v1"
Tooling	88	"add down migrations scripts"
Bug Fixing	71	"category controllers routes bug fixed"
LLM/AI Work	66	"Improved llm-model response handling"
Testing and Verification	39	"test cases for unit testing with code [...]"

(see Section 2.4.1). These clusters illustrate how developers integrated such code in their workflows.

The most frequent activity is *Feature Development* (974 commits), where developers used generated code to add or extend functionality, prototype modules, or implement new features. Example messages such as "user profile added and user slice created" illustrate active feature construction. The second largest category, *Project Initialization and Setup* (723 commits), captures activities related to bootstrapping new projects, configuring environments, and generating scaffolding. For instance, "Initial commit: FileAnalyzer implementation" corresponds to a comment noting, "This file was primarily generated by ChatGPT [shareable ChatGPT link]," indicating LLM involvement in project setup.

Code Refactoring (382 commits) involves restructuring or simplifying existing code for clarity and maintainability, often guided by LLM-generated transformations. For example, "refactoring and adding countries dataset" corresponds to a comment stating, "code created with chatgpt," on a script that scraped country data from Wikipedia.

Documentation (149 commits) includes descriptive improvements such as "Added comments for human and AI code", while *Repository Management* (107 commits) covers activities like migration or restructuring (e.g., "transformed to new git online repo").

Security and Compliance (100 commits) reflects updates enforcing secure or consistent practices ("feat: add new rule 'no-hardcoded-secrets' "). *Release and Deployment* (98 commits)

Table 4: Modification Activities Identified from First Change Commit Messages

Modification Intent	N	Example First Change Commit Message
Code Refactoring	330	"Improved LLM code"
Feature Development	282	"feat: add chat management functionality with MongoDB integration"
Bug Fixing	142	"fix memory leak"
Repository Management	125	"Merge branch 'lock_capsule_feature' into dev"
LLM/AI Work	92	"Edited prompt for punctuation"
Testing and Verification	55	"Fixed Validation"
Documentation	45	"README updated"
Security and Compliance	14	"removed API keys"
Logging	11	"cmd line update / simplify"

includes packaging or publication steps ("stable release v1"), and *Tooling* (88 commits) involves creation or modification of development utilities and automation scripts ("add down migrations scripts").

Smaller categories include *Bug Fixing* (71 commits), which captures corrective changes aimed at resolving functional or logical issues (e.g., "deleteOldImageFromCloudinary method fixed after debugging"). The *LLM/AI Work* category (66 commits) includes commits that integrate or modify LLM-related components within projects, such as prompt handling, model response refinement, or API adjustments (e.g., "Improved llm-model response handling"). Finally, *Testing and Verification* (39 commits) encompasses efforts to validate and ensure correctness of LLM-assisted code through new or updated test cases (e.g., "Added unit test cases for LLM-generated modules").

RQ2.B: (Types of Modifications.) What types of development activities do developers perform on LLM-assisted code after its integration into software projects?

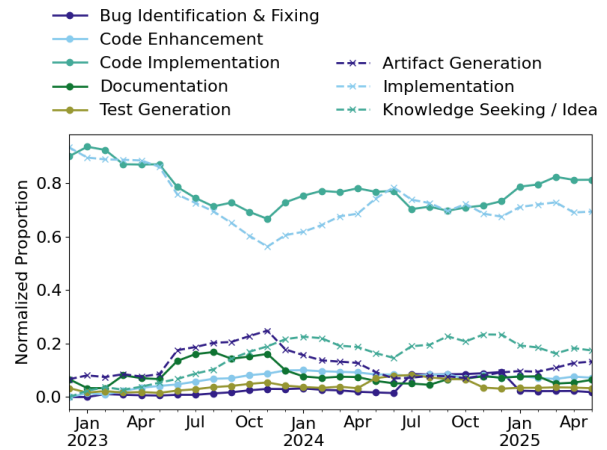
Table 4 summarizes the 9 development activities that were performed on LLM-aided code after its integration into software projects, derived from clustering the *first change commits* (see Section 2.4.2). These clusters capture how developers refined and extended the initially generated files.

The most frequent activities are *Code Refactoring* (330 commits) and *Feature Development* (282 commits), reflecting a shift from introducing generated code to refining and extending it.

Code Refactoring efforts targeted to improve the structure and readability of the LLM-generated code. For example, a *first change commit* message "Improved LLM code" corresponds to a comment "This file contains code generated by GitHub Copilot and edited by a human." indicating that after the initial integration LLM assisted code, it was modified later. Our manual investigation found that the author added logging info in the code and improved functionality.

Feature Development is prevalent in both RQ2.A and RQ2.B. Feature-related changes in RQ2B involved adding or adapting existing functionality (e.g., "feat: add chat management functionality with MongoDB integration").

Bug Fixing (142 commits) and *Repository Management* (125 commits) indicate post-integration stabilization and coordination efforts ("fix memory leak", "Merge branch 'lock_capsule_feature' into dev"). *LLM/AI Work* (92 commits) involves further tuning

**Figure 3: Temporal Evolution of LLM-Assisted Development Task Types and LLM Contribution Types**

of model logic or prompts ("Edited prompt for punctuation"), showing iterative refinement rather than one-time use.

Other observed categories include *Testing and Verification* (55 commits), capturing developers' efforts to ensure correctness and validate functionality (e.g., "Fixed Validation"). *Documentation* (45 commits) reflects improvements in clarity and maintainability as projects matured (e.g., "README updated"). Smaller yet notable groups include *Security and Compliance* (14 commits) and *Logging* (11 commits), indicating activities aimed at reinforcing quality standards and enhancing operational observability.

RQ2 Summary and Discussion. The findings of RQ2 show a sequential pattern in how LLM-assisted code is integrated and evolved within projects. In the initial phase, LLM-generated content appears predominantly in *feature development* and *project setup* commits, indicating that developers use models to implement functionality and rapidly establish project scaffolds. In the subsequent phase, developers focus on *refactoring*, *feature enhancement*, *bug fixing*, and *testing*, reflecting continued human involvement to adapt, verify, and align generated code with project requirements and conventions. This progression illustrates a shift from automated generation to deliberate human oversight, as developers refine and stabilize LLM-produced artifacts to ensure consistency and reliability within the broader development workflow.

3.3 RQ3: How has developers' LLM usage behavior evolved over time?

To answer this RQ, we analyzed the commit data spanning December 2022 to July 2025. The share of LLM-assisted code comments increased sharply over this period: 0.82% in 2022, 18.31% in 2023, 45.58% in 2024, and 35.29% in 2025 (through July). This trajectory indicates rapid adoption between 2023 and 2024, followed by sustained high engagement through 2025. To examine how this usage evolved over time, we conducted a longitudinal analysis.

Longitudinal Analysis of Task Type. *Code Implementation* remained the dominant activity throughout the observation period

(mean = 0.77, SD = 0.13, $\rho_1 = -0.02$), indicating consistent use of LLMs for direct code production. Its weakly negative trend ($\beta = -0.0013$, $p = 0.64$) suggests a stable but slightly declining share over time.

Code Enhancement accounted for a smaller portion of activity (mean = 0.07, SD = 0.04, $\rho_1 = 0.08$) and exhibited a mild positive trend ($\beta = +0.0012$, $p = 0.12$), reflecting gradual growth in refinement and improvement-related tasks. *Documentation* showed a modest average share (mean = 0.09, SD = 0.09, $\rho_1 = 0.06$) with a weak negative trend ($\beta = -0.0016$, $p = 0.43$), indicating relative stability with limited variation over time.

Test Generation displayed low but consistent proportions (mean = 0.04, SD = 0.05, $\rho_1 = 0.09$), following a nearly flat slope ($\beta = +0.0005$, $p = 0.63$), suggesting sustained but minor engagement in validation-oriented use cases. *Bug Identification & Fixing* contributed the smallest share (mean = 0.03, SD = 0.08, $\rho_1 = -0.07$) and showed no significant temporal change ($\beta = +0.0011$, $p = 0.51$).

Longitudinal Analysis of LLM Contribution Type. *Implementation* remained the predominant contribution type across the observation period (mean = 0.71, SD = 0.14, $\rho_1 = 0.26$), showing stable use of LLMs for direct code implementation. The slightly negative trend ($\beta = -0.0035$, $p = 0.25$) indicates a gradual leveling of implementation-focused activity.

Knowledge and Concept Support demonstrated moderate activity (mean = 0.16, SD = 0.10, $\rho_1 = 0.20$) with a clear positive trend ($\beta = +0.0048$, $p = 0.018$), representing the only statistically significant increase among contribution types. This reflects a consistent rise in LLM usage for exploratory and reasoning-oriented interactions. *Artifact Generation*, while less frequent (mean = 0.13, SD = 0.11, $\rho_1 = 0.21$), followed a stable trajectory ($\beta = -0.0013$, $p = 0.56$).

Longitudinal Analysis of Introductory and First Change Commits. When analysis is limited to comments containing both the *introductory* commit and a subsequent *first change* commit, several changes appear. *Code Implementation* remains dominant (mean = 0.76, SD = 0.17, $\rho_1 = 0.20$, $\beta = +0.0018$, $p = 0.64$) but now trends β became upwards, while *Code Enhancement* gains further prominence (mean = 0.10, SD = 0.09, $\rho_1 = 0.28$, $\beta = +0.0031$, $p = 0.10$). *Documentation* becomes volatile (mean = 0.09, SD = 0.13, $\rho_1 = 0.02$, $\beta = -0.0050$, $p = 0.074$), and both *Test Generation* and *Bug Identification & Fixing* remain minor (means ≈ 0.03). Overall ρ_1 weakens, reflecting less persistence and greater temporal variation.

For LLM Contribution types, *Implementation* slightly declines (mean = 0.68, SD = 0.19, $\rho_1 = 0.30$, $\beta = -0.0038$, $p = 0.38$), while *Knowledge and Concept Support* increases (mean = 0.19, SD = 0.13, $\rho_1 = 0.17$, $\beta = +0.0034$, $p = 0.25$). *Artifact Generation* remains steady (mean = 0.13, SD = 0.11, $\rho_1 = 0.19$, $\beta = +0.0004$, $p = 0.87$). Overall, this subset shows higher variability and smoother temporal dynamics.

RQ3 Summary and Discussion. Overall, developers’ LLM usage evolved from implementation-focused activities toward more diversified and mature patterns. While *Code Implementation* remains central, steady growth in *Code Enhancement* and *Knowledge and Concept Support* indicates broader integration of LLMs beyond code generation. Developers increasingly use LLMs for reasoning, refinement, and documentation, reflecting a shift from task automation to cognitive collaboration. Meanwhile, limited changes in *Testing*

and *Bug Fixing* suggest cautious adoption in high-reliability contexts. These trends suggest that LLM-assisted development has evolved from code generation to adaptive collaboration, with LLMs increasingly acting as partners in reasoning, refinement, and helping decision-making.

4 Recommendations

Our findings point to several directions for advancing coding assistants and LLM integration in software development.

From **RQ1.A** and **RQ1.B**, developers use LLMs for a wide range of activities including *implementation*, *documentation*, *testing*, and *bug fixing*. However, a significant and growing portion of their interaction with LLMs involves *Knowledge and Concept Support*. Combined with **RQ3**, which shows that knowledge- and reasoning-oriented usage has been steadily increasing over time, this indicates that developers increasingly treat LLMs not merely as code generators but as *conceptual collaborators*. Future coding assistants should therefore expand beyond producing code to providing adaptive reasoning support—detecting developer intent, surfacing explanations, and addressing knowledge gaps during development. For instance, comment patterns such as “ChatGPT suggested this fix and it seems to work” indicates uncertainty about solution validity, suggesting assistants should also provide brief reasoning or impact explanations within the project context.

Results from **RQ2.A** show that LLMs are primarily integrated during *feature development* and *project setup*, where they accelerate early development. However, **RQ2.B** reveals that this LLM-aided code often requires *subsequent human modification*, mainly through refactoring, debugging, and standardization. This highlights the need for assistants that can *adapt to project context* by leveraging repository history, dependency structure, and coding conventions to produce code that better aligns with existing architectures and reduces manual rework. They could assess generated code against project tests and conventions to prevent integration errors.

Overall, future of LLM-assisted development should emphasize:

- (1) expanding from code generation to *knowledge and reasoning assistance*;
- (2) integrating *project- and repository-specific learning* for context-aware generation; and
- (3) enabling *feedback-driven refinement* where LLMs learn from developer edits.

By addressing these directions, coding assistants can evolve from isolated generators to *adaptive collaborators* that enhance both productivity and understanding throughout the software lifecycle.

5 Related Work

We divide the related work into two parts: (1) code comments and commit messages to infer developer intent and activities, and (2) how LLMs are used in software development.

Code Comments and Commit Messages. Textual artifacts such as code comments and commit messages have long been used to understand developer intent and maintenance behavior [7, 21, 29, 36, 47, 48]. Rani et al. reviewed comment quality, showing that comments capture rationale, design trade-offs, and cognitive processes [41]. Hu et al. found that developers value concise, contextually accurate automated comments [24]. Hindle et al.

classified commit messages into corrective, adaptive, and perfective maintenance categories, showing that message text reflects developer intent and task type [22]. Xue et al. observed that LLMs can produce commit messages comparable to human-written ones, highlighting AI's growing role in developer communication and documentation practices [51], while Katzy et al. investigated how multilingual code comment generation by LLMs introduces unique errors and question the reliability of automatic metrics for evaluating AI-generated comments [30]. Li et al. found that comments presents in AI-generated code increased programmers' adoption, regardless of expertise [31].

LLM Usage in Software Development. Generative AI and coding assistants such as Copilot, ChatGPT, Cursor, and Claude have affected software engineering practices significantly. Barke et al. investigated Copilot usage in real-time editing to characterize collaborative code completion [3]; Du et al. and Jin et al. evaluated model accuracy and usability in code-generation tasks [9, 28]; and Guo et al. examined LLMs' ability to refine or repair code automatically [18]. While these studies demonstrated productivity gains, they captured only short-term and synthetic development scenarios. Mining-based studies shifted toward understanding real-world LLM usage. Grewal et al. analyzed ChatGPT-generated code within GitHub repositories and found that developers adopt and integrate AI-produced snippets into projects [17]. Hao et al. studied 580 ChatGPT conversations shared through pull requests and issues, identifying sixteen inquiry types such as debugging, testing, and documentation [19]. Mohamed et al. and Sagdic et al. analyzed the DevGPT dataset and found that developers use LLMs for programming guidance, framework clarification, and explanation of APIs [34, 43]. At a broader organizational level, Mozannar et al. and Murali et al. observed that AI-assisted programming alters productivity, workload distribution, and cognitive effort in industrial teams [35, 37].

While prior studies have examined LLM usage in software development, limited attention has been given to code comments. We build on this work by analyzing comment- and commit-level data to examine how developers incorporate and refine LLM-generated code in open-source projects.

6 Threats to Validity

Construct validity. Construct validity concerns the extent to which the methods and measures used in the study accurately capture the intended research constructs. Our detection of LLM-related comments relied on keyword-based queries, which may introduce false positives (e.g., unrelated names like "Claude"). We mitigated this by manually validating a random subset for precision. Annotation bias was reduced through independent labeling by multiple annotators, iterative guideline refinement, and inter-annotator agreement checks.

Internal validity. Internal validity refers to whether the observed relationships genuinely reflect causal or meaningful associations, rather than being influenced by confounding factors or methodological artifacts. The linkage between LLM-referenced comments and commits assumes repository histories preserve temporal order and authorship. We excluded commits with anomalous timestamps or inconsistent metadata. Because commits occur at the file level and

may include unrelated changes, we limited analysis to the commit where the LLM-referenced comment first appeared and the earliest subsequent commit modifying the same file.

External validity. External validity concerns how well our findings generalize beyond the specific dataset, repositories, or environments analyzed. Our dataset focuses on public GitHub repositories in Python and JavaScript, which may not represent other ecosystems or closed-source projects. Moreover, it captures only self-admitted LLM usage, explicit mentions of tools like ChatGPT or Copilot, thus underrepresenting silent or unacknowledged use. As LLM-assisted development practices evolve rapidly, our findings should be viewed as representative of current open-source trends rather than the full spectrum of developer behavior.

7 Conclusion and Future Work

This study investigates the range of development tasks where developers rely on LLMs, the forms of contribution these models provide, and how LLM-assisted code is integrated and adapted within real-world software projects. By linking LLM-referenced code comments to their corresponding commits, we provide empirical evidence of how human-AI collaboration unfolds in practice.

Developers use LLMs across diverse software engineering activities, most frequently for implementing new features, generating code snippets, and automating routine tasks such as testing and documentation. Beyond direct code generation, LLMs also contribute conceptually—offering guidance, explanations, and alternative design ideas that help developers reason about implementation choices. Once integrated, however, much of this AI-assisted code undergoes human refinement through refactoring, debugging, and reorganization to meet project-specific standards and maintainability requirements. Together, these patterns reveal an evolving collaboration where LLMs accelerate creation and ideation, while developers continue to ensure reliability and alignment within real-world projects.

Future work should focus on enhancing LLMs' ability to provide adaptive, context-aware assistance that identifies when developers require conceptual support. Equally important directions include assessing the long-term quality and evolution of LLM-generated code, investigating how models can learn from developer modifications, and extending such analyses to larger open-source and industrial ecosystems. Advancing these research fronts can further evolve LLMs into proactive and context-sensitive partners that continuously improve collaboration between automation and human expertise in software engineering.

References

- [1] Eman Abdullah AlOmar, Anushkrishna Venkatakrishnan, Mohamed Wiem Mkaouer, Christian Newman, and Ali Ouni. 2024. How to refactor this code? an exploratory study on developer-chatgpt refactoring conversations. In *Proceedings of the 21st International Conference on Mining Software Repositories*. 202–206.
- [2] Anonymous. 2025. *Replication Package*. <https://anonymous.4open.science/r/oss-llm-3FE7/>
- [3] Shraddha Barke, Michael B James, and Nadia Polikarpova. 2023. Grounded copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111.
- [4] George EP Box, Gwilym M Jenkins, Gregory C Reinsel, and Greta M Ljung. 2015. *Time series analysis: forecasting and control*. John Wiley & Sons.
- [5] Varun Chandola, Arindam Banerjee, and Vipin Kumar. 2009. Anomaly detection: A survey. *ACM computing surveys (CSUR)* 41, 3 (2009), 1–58.

- [6] Moataz Chouchen, Narjes Bessghaier, Mahi Begoug, Ali Ouni, Eman Alomar, and Mohamed Wiem Mkaouer. 2024. How do software developers use chatgpt? an exploratory study on github pull requests. In *Proceedings of the 21st International Conference on Mining Software Repositories*. 212–216.
- [7] Zadia Codabux, Fatemeh Fard, Roberto Verdecchia, Fabio Palomba, Dario Di Nucci, and Gilberto Recupito. 2024. Teaching Mining Software Repositories. In *Handbook on Teaching Empirical Software Engineering*. Springer, 325–362.
- [8] A. Philip Dawid and Allan M. Skene. 1979. Maximum likelihood estimation of observer error-rates using the EM algorithm. *Journal of the Royal Statistical Society: Series C (Applied Statistics)* 28, 1 (1979), 20–28.
- [9] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2024. Evaluating large language models in class-level code generation. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
- [10] Shubhang Shekhar Dvivedi, Vyshnav Vijay, Sai Leela Rahul Pujari, Shoumik Lodh, and Dhruv Kumar. 2024. A comparative analysis of large language models for code documentation generation. In *Proceedings of the 1st ACM international conference on AI-powered software*. 65–73.
- [11] Ramtin Ehsani, Sakshi Pathak, and Preetha Chatterjee. 2025. Towards detecting prompt knowledge gaps for improved llm-guided issue resolution. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 699–711.
- [12] Hamzeh Eyal Salman, Mustafa Hammad, Abdelhak-Djamel Seriai, and Ahed Al-Shoub. 2018. Semantic clustering of functional requirements using agglomerative hierarchical clustering. *Information* 9, 9 (2018), 222.
- [13] Yicheng Gao, Gonghan Xu, Zhe Wang, and Arman Cohan. 2024. Bayesian Calibration of Win Rate Estimation with LLM Evaluators. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Miami, Florida, USA, 4757–4769. doi:10.18653/v1/2024.emnlp-main.273
- [14] Andrew Gelman, John B. Carlin, Hal S. Stern, David B. Dunson, Aki Vehtari, and Donald B. Rubin. 2013. *Bayesian Data Analysis* (3rd ed.). CRC Press.
- [15] GitHub. 2023. The State of Open Source and AI in 2023. <https://github.blog/news-insights/research/the-state-of-open-source-and-ai/> Available Online.
- [16] GitHub. 2024. Octoverse 2024: AI leads Python to top language. <https://github.blog/news-insights/octoverse-2024/> Available Online.
- [17] Balreet Grewal, Wentao Lu, Sarah Nadi, and Cor-Paul Bezemer. 2024. Analyzing developer use of chatgpt generated code in open source github projects. In *Proceedings of the 21st International Conference on Mining Software Repositories*. 157–161.
- [18] Qi Guo, Junming Cao, Xiaofei Xie, Shangqing Liu, Xiaohong Li, Bihuan Chen, and Xin Peng. 2024. Exploring the potential of chatgpt in automated code refinement: An empirical study. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.
- [19] Huizi Hao, Kazi Amit Hasan, Hong Qin, Marcos Macedo, Yuan Tian, Steven H. H. Ding, and Ahmed E. Hassan. 2024. An Empirical Study on Developers’ Shared Conversations with ChatGPT in GitHub Pull Requests and Issues. *Empirical Software Engineering* (2024).
- [20] Zeyu He, Chieh-Yang Huang, Chien-Kuang Cornelia Ding, Shaurya Rohatgi, and Ting-Hao Kenneth Huang. 2024. If in a crowdsourced data annotation pipeline, a gpt-4. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–25.
- [21] Abram Hindle, Daniel M German, and Ric Holt. 2008. What do large commits tell us? a taxonomical study of large commits. In *Proceedings of the 2008 international working conference on Mining software repositories*. 99–108.
- [22] Abram Hindle, Daniel M German, Ric C Holt, and Michael W Godfrey. 2009. Automatic classification of large changes into maintenance categories. In *Proceedings of the 2009 IEEE International Conference on Program Comprehension (ICPC)*. IEEE, 30–39.
- [23] Dirk Hovy, Taylor Berg-Kirkpatrick, Ashish Vaswani, and Eduard Hovy. 2013. Learning whom to trust with MACE. In *Proceedings of NAACL-HLT*. 1120–1130.
- [24] Xing Hu, Xin Xia, David Lo, Zhiyuan Wan, Qiuyuan Chen, and Thomas Zimmermann. 2022. Practitioners’ expectations on automated code comment generation. In *Proceedings of the 44th international conference on software engineering*. 1693–1705.
- [25] Shahana Ibrahim, Panagiotis A Traganitis, Xiao Fu, and Georgios B Giannakis. 2025. Learning from crowdsourced noisy labels: A signal processing perspective. *IEEE Signal Processing Magazine* 42, 3 (2025), 84–106.
- [26] Mia Mohammad Imran, Preetha Chatterjee, and Kostadin Damevski. 2024. Uncovering the causes of emotions in software developer communication using zero-shot llms. In *Proceedings of the IEEE/ACM 46th international conference on software engineering*. 1–13.
- [27] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. [n. d.]. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations*.
- [28] Kailun Jin, Chung-Yu Wang, Hung Viet Pham, and Hadi Hemmati. 2024. Can chatgpt support developers? an empirical evaluation of large language models for code generation. In *Proceedings of the 21st International Conference on Mining Software Repositories*. 167–171.
- [29] Huzefa Kagdi, Michael L Collard, and Jonathan I Maletic. 2007. A survey and taxonomy of approaches for mining software repositories in the context of software evolution. *Journal of software maintenance and evolution: Research and practice* 19, 2 (2007), 77–131.
- [30] Jonathan Katzy, Yongcheng Huang, Gopal-Raj Panchu, Maksym Ziemlewski, Paris Loizides, Sander Vermeulen, Arie van Deursen, and Maliheh Izadi. 2025. A Qualitative Investigation into LLM-Generated Multilingual Code Comments and Automatic Evaluation Metrics. In *Proceedings of the 21st International Conference on Predictive Models and Data Analytics in Software Engineering* (Trondheim, Norway) (PROMISE ’25). Association for Computing Machinery, New York, NY, USA, 31–40.
- [31] Changwen Li, Christoph Treude, and Ofir Turel. 2026. Do comments and expertize still matter? An experiment on programmers’ adoption of AI-generated JavaScript code. *Journal of Systems and Software* 231 (2026), 112634.
- [32] Leland McInnes, John Healy, and Steve Astels. 2017. hdbscan: Hierarchical density based clustering. *Journal of Open Source Software* 2, 11 (2017), 205. doi:10.21105/joss.00205
- [33] Mistral AI. 2024. Mistral AI Models Overview. https://docs.mistral.ai/getting-started/models/models_overview/ Available Online.
- [34] Suad Mohamed, Abdullah Parvin, and Esteban Parra. 2024. Chatting with AI: Deciphering Developer Conversations with ChatGPT. In *Proceedings of the 21st International Conference on Mining Software Repositories (MSR)*. ACM.
- [35] Hussein Mozannar, Gagan Bansal, Adam Fournay, and Eric Horvitz. 2024. Reading between the lines: Modeling user behavior and costs in AI-assisted programming. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–16.
- [36] Fangwen Mu, Xiao Chen, Lin Shi, Song Wang, and Qing Wang. 2023. Developer-intent driven code comment generation. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 768–780.
- [37] Vijayaraghavan Murali, Chandra Maddila, Imad Ahmad, Michael Bolin, Daniel Cheng, Negar Ghorbani, Renuka Fernandez, Nachiappan Nagappan, and Peter C Rigby. 2024. AI-assisted Code Authoring at Scale: Fine-tuning, deploying, and mixed methods evaluation. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1066–1085.
- [38] OpenAI. 2024. Introducing GPT-OSS. <https://openai.com/index/introducing-gpt-oss/> Available Online.
- [39] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirel. 2023. The impact of ai on developer productivity: Evidence from github copilot. *arXiv preprint arXiv:2302.06590* (2023).
- [40] Thomas P Quinn, Ionas Erb, Mark F Richardson, and Tamsyn M Crowley. 2018. Understanding sequencing data as compositions: an outlook and review. *Bioinformatics* 34, 16 (2018), 2870–2878.
- [41] Pooja Rani, Arianna Blasi, Natalia Stulova, Sebastiano Panichella, Alessandra Gorla, and Oscar Nierstrasz. 2023. A decade of code comment quality assessment: A systematic literature review. *Journal of Systems and Software* 195 (2023), 111515.
- [42] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP) and the 9th International Joint Conference on Natural Language Processing (IJCNLP)*. Association for Computational Linguistics, 3982–3992. doi:10.18653/v1/D19-1410
- [43] Ertugrul Sagdic, Arda Bayram, and Md Rakibul Islam. 2024. On the Taxonomy of Developers’ Discussion Topics with ChatGPT. In *Proceedings of the 21st International Conference on Mining Software Repositories (MSR) – Mining Challenge Track*. ACM.
- [44] Tree sitter Contributors. 2024. Tree-sitter: An Incremental Parsing System for Programming Tools. <https://github.com/tree-sitter/tree-sitter> GitHub repository.
- [45] Rion Snow, Brendan O’Connor, Daniel Jurafsky, and Andrew Y. Ng. 2008. Cheap and fast—but is it good? Evaluating non-expert annotations for natural language tasks. In *Proceedings of the 2008 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Honolulu, HI, 254–263.
- [46] Stack Overflow. 2024. Technology | 2024 Stack Overflow Developer Survey. <https://survey.stackoverflow.co/2024/technology> Available Online.
- [47] Daniela Steidl, Benjamin Hummel, and Elmar Juergens. 2013. Quality analysis of source code comments. In *2013 21st international conference on program comprehension (icpc)*. Ieee, 83–92.
- [48] Lin Tan. 2015. Code comment analysis for improving software quality. In *The art and science of analyzing software data*. Elsevier, 493–517.
- [49] John Wilder Tukey et al. 1977. *Exploratory data analysis*. Vol. 2. Springer.
- [50] Jacob Whitehill, Paul Ruvolo, Tingfan Wu, Jacob Bergsma, and Javier R. Movellan. 2009. Whose vote should count more: Optimal integration of labels from labelers of unknown expertise. In *Advances in Neural Information Processing Systems (NeurIPS 22)*. 2035–2043.
- [51] Pengyu Xue, Linhao Wu, Zhongxing Yu, Zhi Jin, Zhen Yang, Xinyi Li, Zhenyu Yang, and Yue Tan. 2024. Automated commit message generation with large

1277	language models: An empirical study and beyond. <i>IEEE Transactions on Software Engineering</i> (2024).	[52] Peiran Yao, Jerin George Mathew, Shehraj Singh, Donatella Firmani, and Denilson Barbosa. 2024. A bayesian approach towards crowdsourcing the truths from llms. In <i>NeurIPS 2024 Workshop on Bayesian Decision-making and Uncertainty</i> .	1335
1278			1336
1279			1337
1280			1338
1281			1339
1282			1340
1283			1341
1284			1342
1285			1343
1286			1344
1287			1345
1288			1346
1289			1347
1290			1348
1291			1349
1292			1350
1293			1351
1294			1352
1295			1353
1296			1354
1297			1355
1298			1356
1299			1357
1300			1358
1301			1359
1302			1360
1303			1361
1304			1362
1305			1363
1306			1364
1307			1365
1308			1366
1309			1367
1310			1368
1311			1369
1312			1370
1313			1371
1314			1372
1315			1373
1316			1374
1317			1375
1318			1376
1319			1377
1320			1378
1321			1379
1322			1380
1323			1381
1324			1382
1325			1383
1326			1384
1327			1385
1328			1386
1329			1387
1330			1388
1331			1389
1332			1390
1333			1391
1334			1392